Chandigarh University

OBJECT-ORIENTED PROGRAMMING (OOP)
MINI PROJECT

BANK MANAGEMENT SYSTEM

using C++

Submitted By

Student Name : Sanchit Chauhan

Roll Number : 25BCD10106

Class : 25BCD-1(A)

Subject : Object Oriented Programming

College Name : Chandigarh University

Teacher's Name : Suman

GitHub Link :

TABLE OF CONTENTS

1. Introduction..... 2

2. Objectives..... 2

3. OOP Concepts Used..... 3

3.1 Class..... 3
3.2 Object..... 3
3.3 Encapsulation..... 4
3.4 Inheritance..... 4
3.5 Polymorphism..... 4
3.6 Abstraction..... 5
3.7 Summary of OOP concepts in the project..... 5

4. Features..... 6

5. Algorithm..... 7

6. Source Code..... 9

7. Sample Output..... 17

7.1 Starting the program and displaying accounts when the bank is empty..... 18
7.2 Creating new accounts..... 18
7.3 Validation while creating an account..... 19
7.4 Depositing money..... 19
7.5 Withdrawing money..... 19
7.6 Checking account details..... 20
7.7 Transferring money..... 20
7.8 Displaying all accounts..... 21
7.9 Invalid amount, invalid menu input and exiting the program..... 21

8. Code Explanation..... 21

8.1 Header files and constants..... 21
8.2 Helper functions..... 22
8.3 Class Account (abstract base class)..... 22
8.4 Classes SavingsAccount and CurrentAccount (derived classes)..... 23
8.5 Class Bank..... 23
8.6 The main() function and objects..... 24
8.7 How balances are updated correctly..... 24
8.8 Common viva questions..... 24

9. Conclusion..... 25

1. Introduction

A **Bank Management System** is a computer program that helps a bank look after its customers' accounts. In a bank, thousands of small tasks are done every day: opening new accounts, accepting deposits, paying out withdrawals, checking balances and transferring money from one account to another. If these tasks are written in registers, they take a lot of time and mistakes are easy to make. A Bank Management System performs the same tasks on a computer, quickly and accurately.

This mini project is a **console-based Bank Management System** written in **C++** using **Object-Oriented Programming (OOP)**. The program shows a simple menu. The bank employee chooses an option, such as creating an account, depositing money or withdrawing money, and the program carries out the task and shows the result on the screen.

The system supports two types of accounts. A **Savings Account** must always keep a minimum balance of Rs. 500. A **Current Account** is allowed to go below zero (overdraft) up to Rs. 5000. Every account is treated as an **object**; it remembers its own account number, holder name and balance, and it knows the rule that applies to its withdrawals. A **Bank** class keeps the list of all accounts and performs the bank operations.

How the system helps to manage bank accounts

- **Keeps proper records:** every account is stored with its number, holder name, type and balance.
- **Updates balances automatically:** every deposit, withdrawal and transfer changes the balance immediately and correctly.
- **Follows bank rules:** a Savings account cannot fall below the minimum balance, and a Current account cannot cross its overdraft limit.
- **Saves time:** account details can be found instantly by account number.
- **Avoids mistakes:** wrong or empty input is rejected, and money cannot be transferred from an account to itself.

2. Objectives

The main objectives of this project are:

1. **To create accounts:** open Savings and Current accounts with a unique account number, holder name and opening deposit.
2. **To deposit money:** add money to an account and update its balance.
3. **To withdraw money:** take money out of an account only when the rules of that account type allow it.
4. **To check account details:** display the holder name, account type and current balance of an account.
5. **To display all accounts:** show a neat table of all accounts with the total money held by the bank.
6. **To transfer money:** move money from one account to another safely.

7. **To apply OOP concepts:** use classes, objects, encapsulation, inheritance, polymorphism and abstraction in a real, working program.
8. **To create a user-friendly program:** provide a menu-driven interface with proper input validation so that wrong input does not crash the program.

3. OOP Concepts Used

Object-Oriented Programming is a way of writing programs in which we think of the program as a group of **objects** that work together, just like real things in the real world. In a bank, the real things are accounts and the bank itself, so they become objects in our program. This section explains each OOP concept in simple words and shows exactly where it is used in the project code.

3.1 Class

A **class** is a blueprint (or template) for creating objects. It describes what data an object will store (data members) and what actions it can perform (member functions). A class by itself does not hold real data; it is only a design.

In this project: four classes are created – `Account` (a general bank account), `SavingsAccount`, `CurrentAccount` and `Bank` (the bank that manages all the accounts).

```
class Bank
{
private:
    vector<Account*> accounts;     // list of all accounts
    int nextAccountNumber;         // number given to the next account
public:
    void createAccount();          // functions that work on the accounts
    void depositMoney();
    // ... other functions ...
};
```

(Shortened view – the full class is in Section 6.)

3.2 Object

An **object** is a real, usable copy of a class. If a class is a blueprint of a house, an object is the actual house built from it. Many objects can be made from one class, and each object keeps its own data.

In this project: `bank` is an object of the `Bank` class created in `main()`. Every time an account is opened, a new `SavingsAccount` or `CurrentAccount` object is created, and each one has its own account number, holder name and balance.

```
Bank bank;                    // object of class Bank (in main)

newAccount = new SavingsAccount(nextAccountNumber, name, opening);
```

3.3 Encapsulation

Encapsulation means wrapping data and the functions that use that data together inside a class, and **hiding the data** from the outside world. Other parts of the program can use the data only through public member functions.

In this project: the data members `accountNumber`, `holderName` and `balance` of the `Account` class are declared `protected`. This means they are hidden from the outside code (such as `Bank` and `main()`) and can be used only by the class itself and its child classes. The balance can be changed only through the public functions `deposit()` and `withdraw()`, and it can be read only through `getBalance()`. This protects the money from accidental or wrong changes.

```
class Account
{
protected:         // hidden from outside, visible to child classes
    int accountNumber;
    string holderName;
    double balance;

public:            // safe way to use the data
    void deposit(double amount) { ... }
    virtual bool withdraw(double amount) = 0;
    double getBalance() const { return balance; }
    // ...
};
```

3.4 Inheritance

Inheritance means creating a new class (child or derived class) from an existing class (parent or base class). The child class automatically gets the data and functions of the parent, so we do not have to write them again. It represents an “is-a” relationship.

In this project: `SavingsAccount` and `CurrentAccount` are both derived from `Account`. A savings account *is an* account and a current account *is an* account, so both inherit `accountNumber`, `holderName`, `balance`, `deposit()` and the getter functions. Each child class adds only what is different, which is its own withdrawal rule.

```
class Account { ... };                          // parent (base) class

class SavingsAccount : public Account { ... };   // child class

class CurrentAccount : public Account { ... };   // child class
```

3.5 Polymorphism

Polymorphism means “many forms”. The same function call can behave differently depending on the object that receives it. In C++ this is done with **virtual functions** and function **overriding**: the base class declares a function as `virtual`, and each child class provides its own version of it.

In this project: `withdraw()` is a virtual function of `Account`. `SavingsAccount` overrides it with the rule “the balance must not fall below Rs. 500”, while `CurrentAccount` overrides it

with the rule “the balance must not fall below minus Rs. 5000”. In the `Bank` class, a pointer of the base type `Account*` is used to call `withdraw()`. At run time, C++ automatically runs the correct version for that account. The same happens with `getAccountType()`.

```
// In SavingsAccount
bool withdraw(double amount) override
{
    if (balance - amount < SAVINGS_MIN_BALANCE)
        return false;
    ...
}

// In CurrentAccount
bool withdraw(double amount) override
{
    if (balance - amount < -CURRENT_OVERDRAFT_LIMIT)
        return false;
    ...
}

// In Bank::withdrawMoney()
if (accounts[index]->withdraw(amount))   // correct version runs
```

3.6 Abstraction

Abstraction means showing only the important features and hiding the unnecessary details. It lets us say *what* an object does without worrying about *how* it does it. In C++, an **abstract class** is a class that has at least one **pure virtual function** (a function written with `= 0`). We cannot create objects of an abstract class; it only acts as a common template for its child classes.

In this project: `Account` is an abstract class because `withdraw()` and `getAccountType()` are pure virtual functions. It only says that every account must be able to withdraw money and tell its type; the child classes decide how. Abstraction is also seen in `main()`: the user simply calls `bank.withdrawMoney()` without needing to know how the account is searched, checked and updated inside the class.

```
virtual bool withdraw(double amount) = 0;      // pure virtual function
virtual string getAccountType() const = 0;     // pure virtual function
```

3.7 Summary of OOP concepts in the project

OOP Concept	Where it is used	Benefit in this project
Class	`Account`, `SavingsAccount`, `CurrentAccount`, `Bank`	Organises the program into logical parts.
Object	`bank` in `main()`; each account created with `new`	Each account keeps its own balance and details.
Encapsulation	Protected data members of `Account`; public `deposit()`, `withdraw()`, `getBalance()`	The balance cannot be changed in a wrong way.

OOP Concept	Where it is used	Benefit in this project
Inheritance	`SavingsAccount` and `CurrentAccount` inherit from `Account`	Common code is written only once.
Polymorphism	Virtual `withdraw()` called through an `Account*` pointer	The correct withdrawal rule runs automatically.
Abstraction	Abstract class `Account`; simple `Bank` functions used by `main()`	Hides complex details and defines a common design.

4. Features

The program is menu driven. The following table lists the features and the menu option used to reach each of them.

Feature	Menu Option	Description
Create new account	1	Opens a Savings or Current account. The program gives a new account number automatically (starting from 1001) and takes the holder name and opening deposit.
Deposit money	2	Adds the entered amount to the balance of the chosen account.
Withdraw money	3	Takes money out of an account only if the rule of that account type allows it.
Check account details	4	Shows the account number, holder name, account type and current balance.
Display all accounts	5	Shows all accounts in a table, with the total number of accounts and the total balance.
Transfer money	6	Moves money from a sender account to a receiver account. The sender’s withdrawal rule applies.
Exit	7	Ends the program.

Account types and their rules

Account type	Opening / balance rule	Withdrawal rule
Savings Account	Opening deposit of at least Rs. 500.00	Balance must never fall below Rs. 500.00.
Current Account	Any positive opening deposit	Balance may go down to minus Rs. 5000.00 (overdraft limit).

Additional features

- **Input validation:** account numbers and menu choices must be positive whole numbers, amounts must be positive with at most two decimal places, and names cannot be empty. Wrong input shows an error message and the program asks again.
- **Automatic account numbers:** the program gives each new account a unique number.
- **Safe transfers:** money is deposited into the receiver's account only after it has been successfully withdrawn from the sender's account, and an account cannot transfer money to itself.
- **Clear messages:** the program always tells the user whether an operation was successful or why it failed.

5. Algorithm

The steps below describe how the program works and how the user interacts with it.

Main algorithm

1. **Start** the program.
2. Create a `Bank` object. At the beginning the bank has no accounts.
3. Display the **main menu** (options 1 to 7).
4. Ask the user to enter a choice. If the input is not a positive whole number, show an error message and ask again.
5. According to the choice entered, do one of the following:
   - **1** – Create a new account.
   - **2** – Deposit money.
   - **3** – Withdraw money.
   - **4** – Check account details.
   - **5** – Display all accounts.
   - **6** – Transfer money.
   - **7** – Show a goodbye message and go to the last step.
   - **Any other number** – Show "Invalid choice" and continue.
6. After the selected task is finished, go back to **Step 3** and display the menu again.
7. **Stop** the program when the user chooses option 7.

Option 1 – Create a new account

1. Show the two account types and ask the user to choose 1 (Savings) or 2 (Current). If the choice is not 1 or 2, show an error and return to the menu.
2. Ask for the **account holder name** (it must not be empty).
3. Ask for the **opening deposit** (a positive amount).
4. If the account is a Savings account and the deposit is less than Rs. 500, show an error and return to the menu.

5. Create a `SavingsAccount` or `CurrentAccount` object with the next account number and store it in the bank's list.
6. Display "Account created successfully!" with the account number and increase the next account number by 1.

Option 2 – Deposit money

1. Ask the user to enter the **account number**. If no such account exists, display a message and return to the menu.
2. Ask for the **amount** to deposit.
3. Add the amount to the account balance.
4. Display the amount deposited and the new balance.

Option 3 – Withdraw money

1. Ask the user to enter the **account number**. If no such account exists, display a message and return to the menu.
2. Ask for the **amount** to withdraw.
3. Ask the account to withdraw the amount. A Savings account checks that the balance stays at Rs. 500 or more; a Current account checks that the balance does not fall below minus Rs. 5000.
4. If the withdrawal is allowed, reduce the balance and display the new balance; otherwise display "Withdrawal failed".

Option 4 – Check account details

1. Ask the user to enter the **account number**.
2. If the account is found, display its number, holder name, type and balance; otherwise display "No account found".

Option 5 – Display all accounts

1. If there are no accounts, display "No accounts have been created yet." and return to the menu.
2. Otherwise print the table heading.
3. Print the details of every account one by one and add up the balances.
4. Print the total number of accounts and the total balance.

Option 6 – Transfer money

1. Ask for the **sender** account number and the **receiver** account number. If either account does not exist, display a message and return to the menu.
2. If both numbers are the same, display an error and return to the menu.
3. Ask for the **amount** to transfer.
4. Try to withdraw the amount from the sender account. If it fails, display "Transfer failed".
5. If it succeeds, deposit the same amount into the receiver account and display the success message.

Option 7 – Exit

1. Display the goodbye message and end the program.

6. Source Code

The complete C++ program is given below. It has been compiled and tested with the GNU C++ compiler (`g++`). It uses only basic C++ features and works in any standard C++ compiler such as Code::Blocks, Dev-C++, Visual Studio or an online C++ compiler.

How to compile and run

- Save the code in a file named `bank.cpp`.
- Compile it using the command: `g++ bank.cpp -o bank`
- Run it using: `./bank` (on Windows: `bank.exe`).
- If you use an IDE such as Code::Blocks or Dev-C++, simply paste the code in a new file and press **Compile & Run**.

Program: bank.cpp

```
// ------------------------------------------------------------
//  Bank Management System
//  A simple OOP mini project in C++
//  Concepts used: Class, Object, Encapsulation, Inheritance,
//                 Abstraction, Polymorphism, Constructors
// ------------------------------------------------------------
#include <iostream>   // for cin, cout
#include <string>     // for string
#include <vector>     // for vector (stores many accounts)
#include <iomanip>    // for setw, setprecision (neat output)
#include <sstream>    // for stringstream (checking numeric input)
#include <cmath>      // for floor, fabs
#include <cstdlib>    // for exit()
using namespace std;

// Rules of the bank (fixed values used by the account classes)
const double SAVINGS_MIN_BALANCE = 500.0;       // Savings must keep this
const double CURRENT_OVERDRAFT_LIMIT = 5000.0;  // Current may go below 0

// ------------------------------------------------------------
//  Helper functions for safe input
// ------------------------------------------------------------

// Reads a positive whole number. Keeps asking until input is valid.
int readInt(const string& prompt)
{
    string line;
    int value;
    char extra;
```

```
    while (true)
    {
        cout << prompt;
        if (!getline(cin, line))            // input ended unexpectedly
        {
            cout << "\nInput ended. Exiting program.\n";
            exit(0);
        }

        stringstream ss(line);
        // Valid only if: a number is read, nothing extra follows, value > 0
        if (ss >> value && !(ss >> extra) && value > 0)
            return value;

        cout << "Invalid input! Please enter a positive whole number.\n";
    }
}

// Reads a money amount (positive, at most 2 decimal places).
double readAmount(const string& prompt)
{
    string line;
    double value;
    char extra;

    while (true)
    {
        cout << prompt;
        if (!getline(cin, line))            // input ended unexpectedly
        {
            cout << "\nInput ended. Exiting program.\n";
            exit(0);
        }

        stringstream ss(line);
        if (ss >> value && !(ss >> extra) && value > 0 && value <= 10000000)
        {
            // Check that there are no more than 2 digits after the point
            double paise = value * 100;
            if (fabs(paise - floor(paise + 0.5)) < 0.000001)
                return value;
        }

        cout << "Invalid amount! Enter a positive amount "
             << "(maximum 2 decimal places).\n";
    }
}

// Reads a non-empty line of text. Keeps asking until input is valid.
string readText(const string& prompt)
{
    string line;
```

```
    while (true)
    {
        cout << prompt;
        if (!getline(cin, line))            // input ended unexpectedly
        {
            cout << "\nInput ended. Exiting program.\n";
            exit(0);
        }

        // Accept only if the line has at least one non-space character
        if (line.find_first_not_of(" \t") != string::npos)
            return line;

        cout << "Invalid input! This field cannot be empty.\n";
    }
}

// Prints the heading line of the account table
void printTableHeader()
{
    cout << left
         << setw(9)  << "Acc No"
         << setw(22) << "Holder Name"
         << setw(10) << "Type"
         << setw(14) << "Balance (Rs.)" << endl;
    cout << string(55, '-') << endl;
}

// ----------------------------------------------------------------
//  Abstract base class: Account  (ABSTRACTION)
//  It cannot be used directly because withdraw() and
//  getAccountType() are pure virtual functions.
// ----------------------------------------------------------------
class Account
{
protected:                      // visible to child classes only
    int accountNumber;
    string holderName;
    double balance;

public:
    // Constructor: sets the account number, holder name and balance
    Account(int number, string name, double openingBalance)
    {
        accountNumber = number;
        holderName = name;
        balance = openingBalance;
    }

    // Virtual destructor (good practice for base classes)
    virtual ~Account() {}

    // Getter functions (read-only access)
    int getAccountNumber() const { return accountNumber; }
    string getHolderName() const { return holderName; }
    double getBalance() const { return balance; }
```

```
    // Deposit is the same for every account type
    void deposit(double amount)
    {
        balance = balance + amount;
    }

    // Pure virtual functions: every child class MUST define them
    virtual bool withdraw(double amount) = 0;
    virtual string getAccountType() const = 0;

    // Prints one row of the account table
    void displayDetails() const
    {
        cout << left
             << setw(9)  << accountNumber
             << setw(22) << holderName
             << setw(10) << getAccountType()   // child's version is called
             << setw(14) << balance << endl;
    }
};

// ---------------------------------------------------------------
//  Derived class: SavingsAccount  (INHERITANCE)
//  Must always keep a minimum balance.
// ---------------------------------------------------------------
class SavingsAccount : public Account
{
public:
    // Constructor passes its values to the base class constructor
    SavingsAccount(int number, string name, double openingBalance)
        : Account(number, name, openingBalance)
    {
    }

    // Overrides withdraw() (POLYMORPHISM)
    bool withdraw(double amount) override
    {
        if (balance - amount < SAVINGS_MIN_BALANCE)
            return false;           // minimum balance would be broken

        balance = balance - amount;
        return true;
    }

    string getAccountType() const override { return "Savings"; }
};

// ---------------------------------------------------------------
//  Derived class: CurrentAccount  (INHERITANCE)
//  May go below zero up to the overdraft limit.
// ---------------------------------------------------------------
class CurrentAccount : public Account
{
public:
    CurrentAccount(int number, string name, double openingBalance)
        : Account(number, name, openingBalance)
    {
    }
```

```
    // Overrides withdraw() with a different rule (POLYMORPHISM)
    bool withdraw(double amount) override
    {
        if (balance - amount < -CURRENT_OVERDRAFT_LIMIT)
            return false;           // overdraft limit would be crossed

        balance = balance - amount;
        return true;
    }

    string getAccountType() const override { return "Current"; }
};

// ------------------------------------------------------------
//  Class: Bank
//  Stores all accounts and performs every bank operation.
//  Pointers to the base class (Account*) are stored so that both
//  Savings and Current accounts can be kept in one list.
// ------------------------------------------------------------
class Bank
{
private:
    vector<Account*> accounts;      // list of all accounts
    int nextAccountNumber;          // number given to the next account

    // Returns the position of an account, or -1 if not found
    int findIndex(int accNo) const
    {
        for (size_t i = 0; i < accounts.size(); i++)
        {
            if (accounts[i]->getAccountNumber() == accNo)
                return (int)i;
        }
        return -1;
    }

public:
    // Constructor: account numbers start from 1001
    Bank()
    {
        nextAccountNumber = 1001;
    }

    // Destructor: frees the memory of all account objects
    ~Bank()
    {
        for (size_t i = 0; i < accounts.size(); i++)
            delete accounts[i];
    }

    // 1. Create a new account
    void createAccount()
    {
        cout << "Select account type:\n";
        cout << "1. Savings Account (minimum balance Rs. 500.00)\n";
        cout << "2. Current Account (overdraft up to Rs. 5000.00)\n";
        int type = readInt("Enter type (1 or 2): ");
```

```
    if (type != 1 && type != 2)
    {
        cout << "Invalid account type!\n";
        return;
    }

    string name = readText("Enter Account Holder Name: ");
    double opening = readAmount("Enter Opening Deposit (Rs.): ");

    if (type == 1 && opening < SAVINGS_MIN_BALANCE)
    {
        cout << "A Savings account needs an opening deposit of at least "
             << "Rs. " << SAVINGS_MIN_BALANCE << ".\n";
        return;
    }

    Account* newAccount;                        // base class pointer
    if (type == 1)
        newAccount = new SavingsAccount(nextAccountNumber, name, opening);
    else
        newAccount = new CurrentAccount(nextAccountNumber, name, opening);

    accounts.push_back(newAccount);
    cout << "Account created successfully! Account Number: "
         << nextAccountNumber << endl;
    nextAccountNumber++;
}

// 2. Deposit money
void depositMoney()
{
    int accNo = readInt("Enter Account Number: ");
    int index = findIndex(accNo);

    if (index == -1)
    {
        cout << "No account found with number " << accNo << ".\n";
        return;
    }

    double amount = readAmount("Enter Amount to Deposit (Rs.): ");
    accounts[index]->deposit(amount);

    cout << "Rs. " << amount << " deposited successfully. "
         << "New balance: Rs. " << accounts[index]->getBalance() << endl;
}

// 3. Withdraw money
void withdrawMoney()
{
    int accNo = readInt("Enter Account Number: ");
    int index = findIndex(accNo);
```

```
        if (index == -1)
        {
            cout << "No account found with number " << accNo << ".\n";
            return;
        }

        double amount = readAmount("Enter Amount to Withdraw (Rs.): ");

        // The correct withdraw() (Savings or Current) runs automatically
        if (accounts[index]->withdraw(amount))
        {
            cout << "Rs. " << amount << " withdrawn successfully. "
                 << "New balance: Rs. "
                 << accounts[index]->getBalance() << endl;
        }
        else
        {
            cout << "Withdrawal failed! Not enough balance for a "
                 << accounts[index]->getAccountType() << " account.\n";
        }
    }

    // 4. Check account details and balance
    void checkAccount() const
    {
        int accNo = readInt("Enter Account Number: ");
        int index = findIndex(accNo);

        if (index == -1)
        {
            cout << "No account found with number " << accNo << ".\n";
            return;
        }

        cout << "Account found:\n";
        printTableHeader();
        accounts[index]->displayDetails();
    }

    // 5. Display all accounts
    void displayAllAccounts() const
    {
        if (accounts.empty())
        {
            cout << "No accounts have been created yet.\n";
            return;
        }

        double totalMoney = 0;

        printTableHeader();
        for (size_t i = 0; i < accounts.size(); i++)
        {
            accounts[i]->displayDetails();
            totalMoney = totalMoney + accounts[i]->getBalance();
        }
```

```
        cout << string(55, '-') << endl;
        cout << "Total accounts: " << accounts.size()
             << " | Total balance: Rs. " << totalMoney << endl;
    }

    // 6. Transfer money from one account to another
    void transferMoney()
    {
        int fromNo = readInt("Enter Sender Account Number: ");
        int fromIndex = findIndex(fromNo);

        if (fromIndex == -1)
        {
            cout << "No account found with number " << fromNo << ".\n";
            return;
        }

        int toNo = readInt("Enter Receiver Account Number: ");
        int toIndex = findIndex(toNo);

        if (toIndex == -1)
        {
            cout << "No account found with number " << toNo << ".\n";
            return;
        }

        if (fromNo == toNo)
        {
            cout << "Sender and receiver accounts cannot be the same!\n";
            return;
        }

        double amount = readAmount("Enter Amount to Transfer (Rs.): ");

        // Money is taken from the sender first; the sender's rules apply
        if (accounts[fromIndex]->withdraw(amount))
        {
            accounts[toIndex]->deposit(amount);
            cout << "Rs. " << amount << " transferred from account "
                 << fromNo << " to account " << toNo
                 << " successfully!\n";
        }
        else
        {
            cout << "Transfer failed! Not enough balance for a "
                 << accounts[fromIndex]->getAccountType() << " account.\n";
        }
    }
};
```

```
// Shows the main menu
void showMenu()
{
    cout << "\n===== BANK MANAGEMENT SYSTEM =====\n";
    cout << "1. Create New Account\n";
    cout << "2. Deposit Money\n";
    cout << "3. Withdraw Money\n";
    cout << "4. Check Account Details\n";
    cout << "5. Display All Accounts\n";
    cout << "6. Transfer Money\n";
    cout << "7. Exit\n";
    cout << "==================================\n";
}

// ------------------------------------------------------------
//  main(): creates the Bank object and runs the menu loop
// ------------------------------------------------------------
int main()
{
    cout << fixed << setprecision(2);   // always show 2 decimal places

    Bank bank;                  // object of class Bank
    int choice;

    do
    {
        showMenu();
        choice = readInt("Enter your choice: ");
        cout << endl;

        switch (choice)
        {
            case 1: bank.createAccount();      break;
            case 2: bank.depositMoney();       break;
            case 3: bank.withdrawMoney();      break;
            case 4: bank.checkAccount();       break;
            case 5: bank.displayAllAccounts(); break;
            case 6: bank.transferMoney();      break;
            case 7:
                cout << "Thank you for using the Bank Management System. "
                     << "Goodbye!\n";
                break;
            default:
                cout << "Invalid choice! "
                     << "Please enter a number between 1 and 7.\n";
        }
    } while (choice != 7);

    return 0;
}
```

7. Sample Output

This section shows a sample run of the program in a console window. The output was produced by running the exact source code given in Section 6. The text typed by the user appears after each prompt (for example, after “Enter your choice: ”).

Note: the main menu is displayed after every operation. To save space, it is shown in full only once (in Section 7.1) and is later replaced by the line `[ ...main menu is displayed again... ]`.

7.1 Starting the program and displaying accounts when the bank is empty

The program starts with the main menu. Option 5 is chosen before any account is created, so the bank is empty.

```
===== BANK MANAGEMENT SYSTEM =====
1. Create New Account
2. Deposit Money
3. Withdraw Money
4. Check Account Details
5. Display All Accounts
6. Transfer Money
7. Exit
================================
Enter your choice: 5

No accounts have been created yet.
```

7.2 Creating new accounts

Option 1 creates a Savings account (number 1001) for Rahul Sharma and a Current account (number 1002) for Priya Verma.

```
[ ...main menu is displayed again... ]
Enter your choice: 1

Select account type:
1. Savings Account (minimum balance Rs. 500.00)
2. Current Account (overdraft up to Rs. 5000.00)
Enter type (1 or 2): 1
Enter Account Holder Name: Rahul Sharma
Enter Opening Deposit (Rs.): 5000
Account created successfully! Account Number: 1001

[ ...main menu is displayed again... ]
Enter your choice: 1

Select account type:
1. Savings Account (minimum balance Rs. 500.00)
2. Current Account (overdraft up to Rs. 5000.00)
Enter type (1 or 2): 2
Enter Account Holder Name: Priya Verma
Enter Opening Deposit (Rs.): 2000
Account created successfully! Account Number: 1002
```

7.3 Validation while creating an account

A Savings account with an opening deposit of Rs. 300 is rejected because the minimum is Rs. 500, and an invalid account type (3) is rejected.

```
[ ...main menu is displayed again... ]
Enter your choice: 1

Select account type:
1. Savings Account (minimum balance Rs. 500.00)
2. Current Account (overdraft up to Rs. 5000.00)
Enter type (1 or 2): 1
Enter Account Holder Name: Amit Singh
Enter Opening Deposit (Rs.): 300
A Savings account needs an opening deposit of at least Rs. 500.00.

[ ...main menu is displayed again... ]
Enter your choice: 1

Select account type:
1. Savings Account (minimum balance Rs. 500.00)
2. Current Account (overdraft up to Rs. 5000.00)
Enter type (1 or 2): 3
Invalid account type!
```

7.4 Depositing money

Option 2 deposits Rs. 1500 into account 1001, so its balance becomes Rs. 6500.00.

```
[ ...main menu is displayed again... ]
Enter your choice: 2

Enter Account Number: 1001
Enter Amount to Deposit (Rs.): 1500
Rs. 1500.00 deposited successfully. New balance: Rs. 6500.00
```

7.5 Withdrawing money

Option 3 withdraws money. The Savings account allows Rs. 2000 but refuses Rs. 4200 because the balance would fall below Rs. 500. The Current account allows Rs. 6000 (balance becomes -4000.00 using the overdraft) but refuses another Rs. 2000 because the overdraft limit of Rs. 5000 would be crossed.

```
[ ...main menu is displayed again... ]
Enter your choice: 3

Enter Account Number: 1001
Enter Amount to Withdraw (Rs.): 2000
Rs. 2000.00 withdrawn successfully. New balance: Rs. 4500.00

[ ...main menu is displayed again... ]
Enter your choice: 3

Enter Account Number: 1001
Enter Amount to Withdraw (Rs.): 4200
Withdrawal failed! Not enough balance for a Savings account.
```

```
[ ...main menu is displayed again... ]
Enter your choice: 3

Enter Account Number: 1002
Enter Amount to Withdraw (Rs.): 6000
Rs. 6000.00 withdrawn successfully. New balance: Rs. -4000.00

[ ...main menu is displayed again... ]
Enter your choice: 3

Enter Account Number: 1002
Enter Amount to Withdraw (Rs.): 2000
Withdrawal failed! Not enough balance for a Current account.
```

7.6 Checking account details

Option 4 shows the details of account 1001 and reports an error for an account number that does not exist (9999).

```
[ ...main menu is displayed again... ]
Enter your choice: 4

Enter Account Number: 1001
Account found:
Acc No   Holder Name          Type      Balance (Rs.)
-------------------------------------------------------
1001     Rahul Sharma         Savings   4500.00

[ ...main menu is displayed again... ]
Enter your choice: 4

Enter Account Number: 9999
No account found with number 9999.
```

7.7 Transferring money

Option 6 transfers Rs. 1000 from account 1001 to account 1002. Transferring from an account to itself is rejected.

```
[ ...main menu is displayed again... ]
Enter your choice: 6

Enter Sender Account Number: 1001
Enter Receiver Account Number: 1002
Enter Amount to Transfer (Rs.): 1000
Rs. 1000.00 transferred from account 1001 to account 1002 successfully!

[ ...main menu is displayed again... ]
Enter your choice: 6

Enter Sender Account Number: 1001
Enter Receiver Account Number: 1001
Sender and receiver accounts cannot be the same!
```

7.8 Displaying all accounts

Option 5 displays all accounts. The balances reflect the transfer: 1001 has 3500.00 and 1002 has -3000.00, so the total balance is Rs. 500.00.

```
[ ...main menu is displayed again... ]
Enter your choice: 5

Acc No   Holder Name          Type      Balance (Rs.)
-------------------------------------------------------
1001     Rahul Sharma         Savings   3500.00
1002     Priya Verma          Current   -3000.00
-------------------------------------------------------
Total accounts: 2 | Total balance: Rs. 500.00
```

7.9 Invalid amount, invalid menu input and exiting the program

Invalid amounts (“abc” and -50) are rejected before a valid amount (250.50) is accepted. An invalid menu input (“x”) and an invalid choice (9) are handled, and then option 7 ends the program.

```
[ ...main menu is displayed again... ]
Enter your choice: 2

Enter Account Number: 1001
Enter Amount to Deposit (Rs.): abc
Invalid amount! Enter a positive amount (maximum 2 decimal places).
Enter Amount to Deposit (Rs.): -50
Invalid amount! Enter a positive amount (maximum 2 decimal places).
Enter Amount to Deposit (Rs.): 250.50
Rs. 250.50 deposited successfully. New balance: Rs. 3750.50

[ ...main menu is displayed again... ]
Enter your choice: x
Invalid input! Please enter a positive whole number.
Enter your choice: 9

Invalid choice! Please enter a number between 1 and 7.

[ ...main menu is displayed again... ]
Enter your choice: 7

Thank you for using the Bank Management System. Goodbye!
```

8. Code Explanation

This section explains the important parts of the program in simple language.

8.1 Header files and constants

Header	Purpose
`<iostream>`	Gives `cin` and `cout` for input and output.
`<string>`	Gives the `string` type to store text such as names.

Header	Purpose
`<vector>`	Gives `vector`, a resizable list that stores all the accounts.
`<iomanip>`	Gives `setw()` and `setprecision()` to print neat columns and two decimal places.
`<sstream>`	Gives `stringstream`, used to check that the user typed a valid number.
`<cmath>`	Gives `floor()` and `fabs()`, used to check that an amount has at most two decimal places.
`<cstdlib>`	Gives `exit()`, used to stop the program safely if the input ends unexpectedly.

Two constants store the bank rules: `SAVINGS_MIN_BALANCE` (Rs. 500) and `CURRENT_OVERDRAFT_LIMIT` (Rs. 5000). Keeping them in one place makes the rules easy to change.

8.2 Helper functions

- **`readInt()`** – shows a prompt, reads a whole line and accepts it only if it is a positive whole number with no extra characters. Otherwise it prints an error and asks again. This is used for menu choices and account numbers.
- **`readAmount()`** – works like `readInt()` but accepts a positive decimal amount (up to 10000000) with at most two digits after the decimal point.
- **`readText()`** – reads a line of text and accepts it only if it contains at least one non-space character, so names cannot be empty.
- **`printTableHeader()`** – prints the heading line of the account table.
- **`showMenu()`** – prints the main menu.

8.3 Class Account (abstract base class)

`Account` stores the information that is common to every bank account.

Data member	Type	Meaning
`accountNumber`	int	Unique number of the account.
`holderName`	string	Name of the account holder.
`balance`	double	Money currently in the account (can be negative for a Current account).

- **Access level:** the data members are `protected`, so they are hidden from outside code but can be used by the child classes.
- **Constructor:** `Account(int number, string name, double openingBalance)` sets the account number, holder name and opening balance when an object is created.

- **Getter functions:** `getAccountNumber()`, `getHolderName()` and `getBalance()` let other classes read the data safely. The word `const` at the end means the function does not change the object.
- **`deposit()`:** adds the amount to the balance. It is the same for every account type, so it is written only once in the base class.
- **Pure virtual functions:** `withdraw()` and `getAccountType()` make the class abstract. Every child class must provide its own version.
- **`displayDetails()`:** prints one row of the table. It calls `getAccountType()`, so the correct type (Savings or Current) is printed for each object.
- **Virtual destructor:** `virtual ~Account() {}` is a good-practice line for base classes that have virtual functions. It makes sure a child object is removed correctly when it is deleted through a base class pointer.

8.4 Classes SavingsAccount and CurrentAccount (derived classes)

Both classes inherit from `Account`. Their constructors pass the values to the base class constructor, for example `SavingsAccount(...) : Account(number, name, openingBalance)`. Each class then overrides the two pure virtual functions, using the keyword `override`.

	SavingsAccount	CurrentAccount
`getAccountType()`	Returns “Savings”.	Returns “Current”.
`withdraw(amount)` fails when	`balance - amount` is less than Rs. 500.	`balance - amount` is less than minus Rs. 5000.
`withdraw(amount)` on success	Reduces the balance and returns `true`.	Reduces the balance and returns `true`.

8.5 Class Bank

The `Bank` class manages all the accounts. It has two data members: `vector<Account*> accounts` (the list of accounts) and `nextAccountNumber` (the number given to the next new account).

- **Why a pointer to Account?** A pointer of the base class type can point to an object of any child class. This lets one vector store both Savings and Current accounts, and it makes the correct `withdraw()` run through polymorphism.
- **`new` and `delete`:** each account is created with `new` when it is opened. The destructor `~Bank()` goes through the list and uses `delete` to free the memory when the program ends.
- **Constructor:** `Bank()` sets `nextAccountNumber` to 1001.
- **`findIndex(accNo)`** (private) – searches the vector and returns the position of the account with that number, or -1 if it is not found. It is used by most of the other functions.

- **`createAccount()`** – reads the account type, name and opening deposit, checks the Savings minimum, creates the correct object with `new` and stores it using `push_back()`.
- **`depositMoney()`, `withdrawMoney()`, `checkAccount()`** – ask for the account number, check that the account exists and then call the matching function of the `Account` object.
- **`displayAllAccounts()`** – prints every account and adds up the balances to show the total.
- **`transferMoney()`** – checks both accounts and the amount, withdraws from the sender (the sender's rules apply) and deposits the same amount into the receiver only if the withdrawal succeeded.

8.6 The main() function and objects

In `main()`, `cout << fixed << setprecision(2);` makes every amount print with two decimal places. One object of the `Bank` class is created: `Bank bank;`. A `do...while` loop displays the menu, reads the user's choice with `readInt()` and uses a `switch` statement to call the matching function of the `bank` object. The loop continues until the user enters 7.

8.7 How balances are updated correctly

The program makes sure that balances are always correct by changing them only in one place and by checking the rules first:

- **Deposit:** `Account::deposit()` is the only function that adds money, and `readAmount()` makes sure the amount is positive.
- **Withdrawal:** the balance is reduced only inside `withdraw()`, and only after the account's rule has been checked. If the rule fails, the function returns `false` and the balance stays unchanged.
- **Transfer:** the receiver is credited only after the sender's `withdraw()` returned `true`, so money is never created or lost.
- **Encapsulation** guarantees that no other code can change `balance` directly.

8.8 Common viva questions

Question	Answer
What is a class and what is an object?	A class is a blueprint; an object is a real copy created from that blueprint. Example: `SavingsAccount` is a class and the account of Rahul Sharma is an object.
Why is the balance not directly accessible?	To protect the data (encapsulation). It can be changed only through `deposit()` and `withdraw()`.
Which class is abstract and why?	`Account`, because it has pure virtual functions (`= 0`). We cannot create an object of it.
Where is inheritance used?	`SavingsAccount` and `CurrentAccount` are derived from `Account` and inherit its data and functions.

Question	Answer
Where is polymorphism used?	The virtual function `withdraw()` is called through an `Account*` pointer, and the Savings or Current version runs depending on the object.
What does the constructor do?	It runs automatically when an object is created and sets the starting values, e.g. the account number, name and opening balance.
Why is a pointer (`Account*`) used?	One vector can then store both Savings and Current accounts, and polymorphism works through the base class pointer.
How does the program stop a Savings account from going below Rs. 500?	The Savings version of `withdraw()` checks `balance - amount` against the minimum balance and returns `false` if the rule would be broken.

9. Conclusion

The Bank Management System developed in this project is a simple, menu-driven C++ program that handles the main tasks of a small bank. It allows the user to create Savings and Current accounts, deposit and withdraw money, check account details, display all accounts and transfer money between accounts. The balance of every account is updated automatically and correctly, the rules of each account type are followed, and the input validation ensures that wrong input does not crash the program.

Usefulness: the system replaces slow and error-prone paper registers with a fast and accurate digital record. It shows how real banking rules, such as a minimum balance and an overdraft limit, can be built into a program, and it can easily be extended in the future with features such as saving data in a file, interest calculation, a password (PIN) for each account and transaction history.

OOP concepts learned: while developing this project we learned how to design a program using **classes and objects**; how **encapsulation** protects data such as the balance; how **inheritance** allows `SavingsAccount` and `CurrentAccount` to reuse the code of `Account`; how **polymorphism** lets a base-class pointer call the correct overridden `withdraw()`; and how **abstraction** is achieved with an abstract class and simple functions that hide the internal details. We also practised constructors, destructors, vectors, string handling and input validation.

Overall, the project shows how OOP makes a program easier to understand, maintain and extend, and it gave us practical experience that will be useful in larger software projects.